

Shop

E-commerce Website — Architecture & Production Guide

A standalone Next.js storefront: catalog, search, cart, Stripe checkout, and buyer / seller accounts — built on managed serverless infrastructure.

Repository: github.com/fawzinajjar/shop

Generated: 30 May 2026

Stack: Next.js 14 · TypeScript · Prisma / PostgreSQL · Cloudflare R2 · Stripe · Auth.js

1. What this is

Shop is a full e-commerce web application designed to serve a large product catalog (built and tested toward ~100,000 products), each with descriptive info and a PNG image. Visitors browse and search the catalog, open product pages, add items to a cart, and check out through Stripe. Registered users have accounts with two roles: **Buyers** (shop and view order history) and **Sellers** (list and manage their own products).

Everything runs on managed, pay-as-you-go services, so there are no servers to maintain and the app scales automatically. The database is **optional on first run** — the app boots and shows a setup banner until one is connected, so a teammate can clone and run it immediately.

2. Technology stack

Layer	Technology	Role
Framework	Next.js 14.2 (App Router)	Server-rendered pages, ISR caching, API routes, server actions
Language	TypeScript 5.7	Type-safe app + data layer
UI	React 18 + plain CSS	Components; brand design tokens in globals.css
Database	PostgreSQL + Prisma 5.22	Catalog, users, orders; type-safe ORM queries
Object storage	Cloudflare R2 (S3 API)	Product PNG assets; zero egress fees behind CDN
Auth	Auth.js / NextAuth v5	Email + password (Credentials), JWT sessions
Password hashing	bcryptjs	Salted password hashes (never plaintext)
Payments	Stripe Checkout 17	Hosted, PCI-compliant payment page + webhook
Cart state	Zustand 5	Client-side cart persisted to localStorage

Layer	Technology	Role
Image processing	sharp	Resize / encode uploaded images to PNG
Test data	Faker + embedded-postgres	Seed script; throwaway local database
Hosting target	Vercel	Serverless deploy, CDN, automatic HTTPS

3. How it works — key flows

Browsing the catalog

The home page `/` is a dynamic server component. On each request it queries the database with **keyset (cursor) pagination** — never `OFFSET`, so page 5,000 is as fast as page 1 — plus the category list, and renders the product grid on the server. Images lazy-load from R2 via CDN.

Product pages

`/product/[slug]` uses **Incremental Static Regeneration**: pages are not prebuilt (an empty `generateStaticParams`), but rendered on first visit and cached for an hour (`revalidate = 3600`). This is what makes 100k products viable — no giant build step.

Search & filtering

Driven by URL query params (`q`, `category`, `sort`, `cursor`). Search matches title/description; in production a `pg_trgm` GIN index (shipped in `prisma/search-index.sql`) makes it fast. Price sorts use a composite cursor so pagination stays correct.

Cart & checkout

- The cart lives entirely in the browser (Zustand + localStorage) — no server session needed to shop.
- `POST /api/checkout` **recomputes every price from the database** (it never trusts the client), creates a Stripe Checkout Session, writes a *pending* Order (linked to the account if signed in), and returns the hosted payment URL.
- After payment, Stripe redirects to `/success` and calls `POST /api/webhooks/stripe`. That endpoint **verifies the Stripe signature** and flips the Order to *paid* (or *failed*).

Accounts & roles

- `/register` and `/login` post to server actions; NextAuth's Credentials provider checks the bcrypt hash and issues a signed JWT session.
- The role (Buyer or Seller) is chosen at signup and carried in the JWT, so route guards work without a database lookup on every request.
- `/seller` is a seller-only dashboard to create / edit / delete products with image upload (sharp, then to R2, or local disk in dev). Listings appear in the catalog and on the public storefront `/sellers/[id]`.

Resilience: optional database

Every read path detects an unreachable or unconfigured database (`isDbConnectionError`) and degrades gracefully: the home page shows a “No database connected” banner, product/seller pages return 404, and login fails cleanly — instead of crashing with an unhandled error.

4. Route map

Path	Type	Purpose
/	Dynamic SSR	Catalog: search, filter, keyset pagination
/product/[slug]	ISR (cached)	Product detail + add to cart
/cart	Client	Cart contents; triggers checkout
/success	Client	Post-payment confirmation; clears cart
/login · /register	Client + action	Sign in / sign up (Buyer or Seller)
/account	Dynamic (auth)	Profile + order history
/seller	Dynamic (seller)	Seller product dashboard
/seller/products/new	Dynamic (seller)	Create a product (with image upload)
/seller/products/[id]/edit	Dynamic (seller)	Edit / delete a product
/sellers/[id]	Dynamic (public)	Public seller storefront
/api/checkout	Route handler	Create Stripe session + pending order
/api/webhooks/stripe	Route handler	Mark orders paid (Stripe webhook)
/api/auth/[...nextauth]	Route handler	Auth.js sign-in / session endpoints

5. Data model (Prisma / PostgreSQL)

Model	Key fields	Notes
Product	id, seq, slug*, title, description, priceCents, currency, category, attributes(Json), sellerId?	seq autoincrement powers keyset paging; indexed on (category, seq), (priceCents), (sellerId, seq)
ProductImage	productId, r2Key, alt, position	Image objects stored in R2; ordered per product
User	id, email*, passwordHash, name, role	role = BUYER SELLER; password is bcrypt-hashed
SellerProfile	userId*, storeName, bio, avatarKey?	One per seller; shown on the storefront
Order	id, email, userId?, status, totalCents, currency, stripeSessionId*	Created pending at checkout; webhook marks paid
OrderItem	orderId, productId, title, priceCents, quantity	Price snapshot at purchase time

* = unique. Relations: a Seller (User) has many Products; a Product has many Images; an Order has many OrderItems and may link to a User.

6. Project layout

```
app/ Routes (App Router)
page.tsx Catalog (home)
product/[slug]/ Product detail (ISR)
cart/ success/ Cart + post-payment
(auth)/login, register Auth pages + server actions
account/ Buyer profile + orders
seller/ sellers/[id]/ Seller dashboard + public storefront
api/checkout, api/webhooks/stripe, api/auth/[...nextauth]
lib/ db, products, r2, cart, stripe, money, session, upload
prisma/ schema.prisma, search-index.sql (pg_trgm)
scripts/ seed.ts (test data), _pgdev.mjs (local DB)
auth.ts NextAuth config (Credentials, JWT)
next.config.mjs Image hosts (R2) + config
```

7. Running it locally

```
npm install
cp .env.example .env # set DATABASE_URL; R2/Stripe optional

# No Postgres installed? Start a throwaway one (separate terminal):
npm run db:local # postgres @ localhost:5433

npx prisma db push # create tables
psql "$DATABASE_URL" -f prisma/search-index.sql # fast search index
SEED_COUNT=500 npm run seed # test products (omit for 100,000)

npm run dev # http://localhost:3000
```

Without a database the app still starts and shows a setup banner.

8. What's needed for live production

The app deploys to **Vercel** and depends on three managed services (database, object storage, payments) plus a couple of secrets. Steps:

8.1 Hosting — Vercel

- Import the GitHub repo into Vercel. The build command is `prisma generate && next build` (already in `package.json`).
- Set all environment variables (table below) in the Vercel project settings.
- Add a custom domain; Vercel provisions TLS/HTTPS automatically.

8.2 Database — Neon (serverless Postgres)

- Create a Neon project and copy the **pooled** connection string into `DATABASE_URL`.
- Apply the schema: `npx prisma migrate deploy` (recommended for prod) or `prisma db push`.
- Run `prisma/search-index.sql` once to create the `pg_trgm` GIN search indexes.
- Seed real data (or run the seed once) so the catalog isn't empty.

8.3 Object storage — Cloudflare R2

- Create an R2 bucket and enable public access (`r2.dev` URL) or bind a custom CDN domain.
- Set `R2_ACCOUNT_ID`, `R2_ACCESS_KEY_ID`, `R2_SECRET_ACCESS_KEY`, `R2_BUCKET`, and `R2_PUBLIC_URL` (no trailing slash).
- Add the public image hostname to `images.remotePatterns` in `next.config.mjs`.

8.4 Payments — Stripe

- Set `STRIPE_SECRET_KEY` (test key first, then live).
- Add a webhook endpoint `https://your-domain/api/webhooks/stripe` and put its signing secret in `STRIPE_WEBHOOK_SECRET`.
- Locally, forward events with `stripe listen --forward-to localhost:3000/api/webhooks/stripe`.

8.5 Auth & site URL

- Generate a strong `AUTH_SECRET` with `openssl rand -base64 32`.
- Set `NEXT_PUBLIC_SITE_URL` to the production origin (used to build Stripe success / cancel URLs).

Environment variables

Variable	Needed for	Purpose
<code>DATABASE_URL</code>	All data	PostgreSQL connection (use the pooled string)
<code>AUTH_SECRET</code>	Accounts	Signs / verifies JWT session tokens

Variable	Needed for	Purpose
NEXT_PUBLIC_SITE_URL	Checkout	Builds Stripe success & cancel URLs
STRIPE_SECRET_KEY	Payments	Stripe API key (server-side)
STRIPE_WEBHOOK_SECRET	Orders	Verifies incoming webhook signatures
R2_ACCOUNT_ID	Images	Cloudflare R2 account
R2_ACCESS_KEY_ID	Images	R2 access key
R2_SECRET_ACCESS_KEY	Images	R2 secret key
R2_BUCKET	Images	R2 bucket name
R2_PUBLIC_URL	Images	Public bucket / CDN base URL

Secrets live only in environment variables — `.env` is git-ignored and must never be committed.

9. Hardening checklist (before real customers)

The current build is a solid MVP. For a real launch, plan for:

- **Migrations:** use versioned `prisma migrate` rather than `db push`.
- **Search at scale:** the `pg_trgm` index covers most needs; consider Typesense / Algolia for instant typo-tolerant search on very large catalogs.
- **Email:** order receipts and password flows via a transactional email provider.
- **Commerce:** inventory / stock, tax (Stripe Tax), and shipping rates.
- **Security:** rate-limiting and stricter validation on API routes; size + type limits and moderation on seller image uploads.
- **Reliability:** error tracking (e.g. Sentry), logging, uptime + analytics.
- **Quality:** automated tests and a CI pipeline.
- **Content:** replace seeded placeholder images with real product assets.

10. Cost & scale

Rough MVP cost: Vercel Hobby/Pro + Neon free/launch tier + Cloudflare R2 (~\$0.015 / GB stored, **no egress fees**) + Stripe per-transaction. Effectively near-zero until meaningful traffic.

Scale techniques already built in: keyset pagination (constant-time deep pages), ISR caching for product pages, images served from R2 + CDN, and server-side price recomputation at checkout. The design targets a 100,000-product catalog.